SparseDR: Differentiable Rendering of Sparse Signed Distance Fields

Alexey Budak[1,2] , **Albert Garifullin**[2] , **Vladimir Frolov**[1,2]
[1]IAI, Lomonosov Moscow State University
[2]Lomonosov Moscow State University
{ alexey.budak, albert.garifullin, vladimir.frolov }@graphics.cs.msu.ru

Abstract

We present SparseDR, a novel differentiable rendering algorithm designed for sparse representations based on Signed Distance Fields (SDF). We leverage the Sparse Brick Set representation and propose an adaptation of redistancing for sparse SDFs. This enables SparseDR to surpass existing works in accuracy by increasing the effective resolution of the SDF representation. SparseDR is efficiently implemented in C++ and Vulkan, achieving several times shorter reconstruction time than other methods.

1 Introduction

The signed distance function (SDF) provides a continuous implicit representation of geometry, defining the shortest Euclidean distance from any point in space to the nearest surface, with the sign indicating interior and exterior regions. It has proved to be a convenient shape representation for differentiable rendering, allowing robust and relatively simple gradient-based optimization [Wang *et al.*, 2024].

In an SDF-based differentiable rendering, each gradient descent step updates the distance values at grid cells, so the grid no longer strictly satisfies the SDF property. To address this issue, existing methods typically apply *redistancing* – a process that recomputes grid values so that each cell stores the correct distance to the surface [Detrixhe *et al.*, 2013]. This makes SDF-based differentiable rendering on sparse data structures non-trivial, so all existing methods rely on regular grids. Unfortunately, this approach results in high memory and computational costs, while the reconstruction accuracy remains limited by the maximum grid resolution. Our work proposes a solution to this problem, alleviating limitations through better data structures and sampling techniques:

- We propose an adaptation of SDF-based differentiable rendering to a sparse data structure, which increases the effective resolution of the SDF representation and thereby improves the surface reconstruction accuracy.
- We adapt redistancing to sparse data structures by separating it into two distinct algorithms: local and global redistancing.

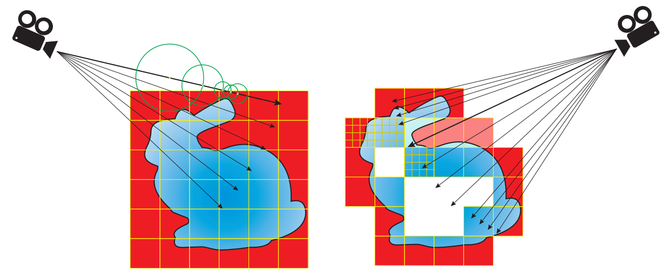

Figure 1: Existing methods use a dense SDF (left) grid to represent the surface, sphere tracing for ray-scene intersection, and uniform image sampling when shooting rays. SparseDR (on the right) uses Sparse Brick Set, where a ray intersects each brick on its way until it hits the surface. It also uses adaptive sampling for boundary integral, placing more rays in the relaxed boundary area. Red color depicts positive SDF values, and blue color depicts negative ones.

2 Related Work

The representation of SDF on a regular grid in existing works [Vicini *et al.*, 2022; Wang *et al.*, 2024; Zhang *et al.*, 2025] has three major limitations: (1) significant memory consumption, especially when using automatic differentiation frameworks such as DrJit [Jakob *et al.*, 2022] or PyTorch [Jason and Yang, 2024]; (2) slow ray–surface intersection based on sphere tracing; (3) an expensive redistancing algorithm that propagates modified SDF values across the entire grid. Our work shows how these components – data structure, ray intersection, and redistancing – can be joined together in an efficient manner.

3 Proposed Method

SparseDR uses the Sparse Brick Set (SBS) proposed in [Söderlund *et al.*, 2022], a hybrid of hierarchical and regular representations (Figure 1). The entire scene is encoded as a BVH, while its leaves store small regular SDF grids, for example $4 \times 4 \times 4$ voxels. For gradient computation, we adapt the relaxed boundary method from [Wang *et al.*, 2024], which we refer to as our baseline.

The proposed optimization process consists of several epochs with multiple gradient descent optimization steps in each. At the beginning, the distance function is initialized with a sphere defined on a low-resolution grid (e.g., 32^3). At each iteration we estimate the gradient and update the distance values using a gradient descent step followed by a reg-

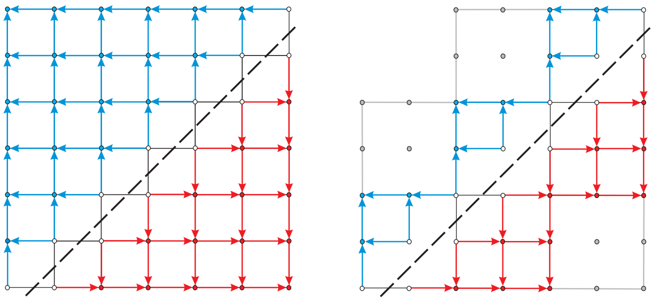

Figure 2: Comparison of the global redistancing algorithm on dense grid (on the left) with the local redistancing (on the right) on SBS, bricks are 3x3 distances in size. The dots represent the distance values. Red and blue dots are the distances outside and inside the object respectively. Dashed line represents the surface, set by distances colored white. Arrows show the order of updating the values. Bricks without surface (grey) are ignored by local redistancing.

ularization and local redistancing (see Figure 2). At the end of each epoch two steps are executed: upsampling and global redistancing. After the last epoch, only global redistancing is performed.

Gradient evaluation. The key feature of SparseDR is the transition from a regular grid to a Sparse Brick Set. A BVH tree is constructed for the SBS, with each leaf node storing a single brick. Instead of sphere tracing, we have adapted a faster Newton method for ray–SDF intersection [Söderlund *et al.*, 2022]. A ray first traverses the BVH; if an intersection with a leaf node is found, it then intersects the voxels of the brick. The ray traverses each brick along its path until it hits the surface or there are no bricks left. The Newton method was modified to search for the relaxed boundary points: within the voxel, a potential relaxed boundary point is found as the root of a quadratic equation, after which the SDF value at the point is checked to be below the relaxed epsilon. The case when the local minimum occurs on the voxel boundary is handled separately.

SparseDR is implemented in C++ and Vulkan and does not use automatic differentiation, i.e., all derivatives were obtained analytically. In the shader we accumulate the pixel color in batches of 16 samples, storing the information about voxels the samples hit in a fixed-size array. Every 16 samples, or after the last one, the gradients are backpropagated using atomic operations.

SparseDR optionally uses an additional pass that only samples pixels containing boundaries and evaluates the boundary integral only, without the need for color computation. This allows for cheaper boundary samples, which helps reduce variance.

Local Redistancing. We implemented two versions of redistancing: local and global (Figure 2). The local version is applied at each iteration, recomputing distances only within bricks containing the surface. In practice, the local redistancing is almost always sufficient, since our ray–surface intersection method relies only on the distances within the traversed bricks. In other words, for the proposed method, the distances in the bricks without a surface are not required to strictly satisfy the SDF property during optimization. However, this approach may cause distance inconsistencies between adjacent bricks, so we perform global redistancing at the end of each epoch.

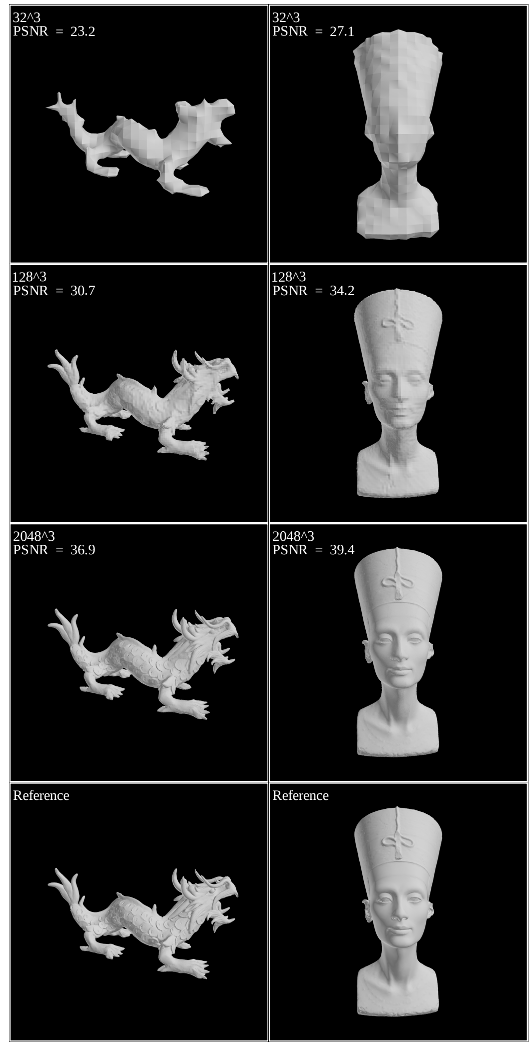

Figure 3: Reconstructioin examples for different SBS resolutions.

Upsampling and Global Redistancing. The global redistancing addresses distance updates across empty space between bricks via extrapolation between brick faces. For every face lacking a direct neighbor, the nearest neighbor bricks are found, and one of their faces is selected for extrapolation. These neighbors are determined once; afterwards, values are extrapolated after each fast sweeping step [Detrixhe *et al.*, 2013]. From extrapolated and prior distances, the number with the smaller absolute value is chosen. The fast sweeping is applied as the final step to propagate extrapolated distances.

The upsampling step starts with SBS sparsification. During this stage, only the bricks containing the surface are preserved, as well as all of their neighbors within a radius of 1 brick, diagonal neighbors included. If any of the adjacent bricks are missing, they are added now. This is necessary when the reconstructed object contains fine details so that the bricks where these details' reconstruction should reside

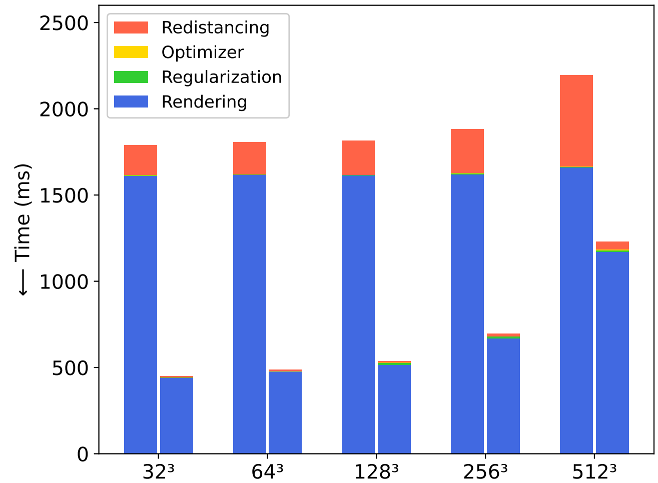

Figure 4: Average time per iteration. For each grid size, the left bar shows the baseline, the right – SparseDR for the SBS of equivalent resolution.

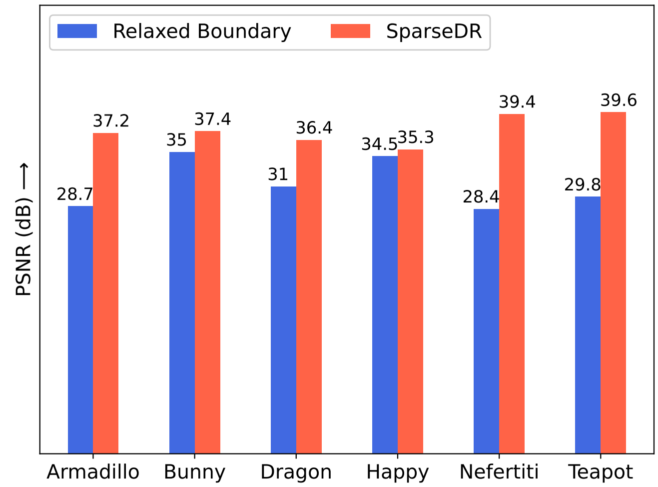

Figure 5: Average PSNR values across viewpoints for models.

Representation	64^3	128^3	512^3	1024^3	2048^3
Baseline	**0.3**	**8.0**	512	4096*	32768*
Ours(Armadillo)	3.1	14.1	**126.2**	**514.2**	**1074.2**
Baseline	**0.3**	**8.0**	512	4096*	32768*
Ours(Bunny)	2.9	11.4	**195.1**	**775.3**	**3093.8**
Baseline	**0.3**	8	512	4096*	32768*
Ours(Dragon)	1.8	**4.8**	**67.9**	**275.6**	**1154.6**
Baseline	**0.3**	8	512	4096*	32768*
Ours(Happy)	1.5	**4.9**	**88.5**	**373.9**	**1562.8**
Baseline	**0.3**	8	512	4096*	32768*
Ours(Nefertiti)	2.2	**7.3**	**114.1**	**459.0**	**1845.3**
Baseline	**0.3**	8	512	4096*	32768*
Ours(Teapot)	1.7	**6.3**	**96.4**	**392.1**	**1597.7**

Table 1: Model sizes for Relaxed Boundary and SparseDR methods. All values are in MB. The baseline values with asterisk are theoretical, as we could not aquire them on our hardware (OOM in Dr.JIT).

would otherwise be absent. Thus, this step not only contributes to reducing the model size but also eliminates a potential limitation compared to the baseline. The upsampling doubles the resolution of the remaining bricks, generating eight new bricks in place of each original one. The new distance values are computed using trilinear interpolation. After upsampling, the global redistancing is applied to recompute all distances, ensuring that the model represents a valid SDF.

4 Experiments

Experiments with the baseline were conducted on the official implementation [Wang, 2024] original setup. The only modifications are the number of viewpoints, the batch size, and the epoch sizes. For the consistency across experiments, an identical lighting setup was used: two directional light sources, $(1, 1, 1)$ and $(-1, -1, -1)$, and the ambient light. The experiments were carried out on six models. 4 models are from the Stanford 3D scanning repository: Armadillo, Asian Dragon, Stanford Bunny, and Happy Buddha. The other two are Nefertiti scan [Nelles and Al-Badri, 2015] and the Utah Teapot. Reconstruction used 16 viewpoints, uniformly distributed over a sphere around the scene, generated by the algorithm from [Wang, 2024]. The batch size is 4 viewpoints, image resolution is 1024×1024.

In our implementation the full optimization process comprised 1000 steps, with upsampling applied at steps 200, 400, 600, 700, 800, and 900. Hardware configuraiton is AMD Ryzen 9 7950X CPU and NVIDIA GeForce RTX 4090 GPU. Some reconstruction examples are shown in Figure 3.

Figure 4 shows the average reconstruction time across all models for each epoch for SparseDR and the baseline. The models are labeled by the SDF grid of corresponding resolution. "Rendering" refers to consecutive rendering of all views within the current batch. Figure 5 presents the mean reconstruction quality across viewpoints for SparseDR and the baseline. Finally, Table 1 illustrates the model sizes for SparseDR and the baseline obtained after each epoch. For the baseline it shows the grid size, for SparseDR both the size of the SBS and of its BVH. The results show increased reconstruction speed and better quality compared to the baseline, while the increase of the SBS size per epoch is an order of magnitude lower than that of the SDF grid.

5 Conclusion

In this work, we present SparseDR, an SDF-based differentiable rendering method using sparse SDF representations. Building on [Wang *et al.*, 2024] as our baseline, we leverage SBS advantages to outperform the original approach while reducing memory usage by an order of magnitude. Future work will focus on performance improvements. We believe the current implementation offers substantial potential for efficiency gains via more accurate gradient computation and reduced per-pixel samples.

References

[Detrixhe *et al.*, 2013] Miles Detrixhe, Frédéric Gibou, and Chohong Min. A parallel fast sweeping method for the eikonal equation. *Journal of Computational Physics*, 237:46–55, 2013.

[Jakob *et al.*, 2022] Wenzel Jakob, Sébastien Speierer, Nicolas Roussel, and Delio Vicini. Dr.jit: a just-in-time compiler for differentiable rendering. 41(4), 2022.

[Jason and Yang, 2024] Jason and Edward He et al. Yang. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '24, page 929–947, New York, NY, USA, 2024. Association for Computing Machinery.

[Nelles and Al-Badri, 2015] Jan Nikolai Nelles and Nora Al-Badri. Nefertiti scan, 2015.

[Söderlund *et al.*, 2022] Herman Hansson Söderlund, Alex Evans, and Tomas Akenine-Möller. Ray tracing of signed distance function grids. *Journal of Computer Graphics Techniques Vol*, 11(3), 2022.

[Vicini *et al.*, 2022] Delio Vicini, Sébastien Speierer, and Wenzel Jakob. Differentiable signed distance function rendering. *ACM Trans. Graph.*, 41(4), July 2022.

[Wang *et al.*, 2024] Zichen Wang, Xi Deng, Ziyi Zhang, Wenzel Jakob, and Steve Marschner. A simple approach to differentiable rendering of sdfs. In *SIGGRAPH Asia 2024 Conference Papers*, New York, NY, USA, 2024. Association for Computing Machinery.

[Wang, 2024] Zichen Wang. A simple approach to differentiable rendering of sdfs repository, 2024.

[Zhang *et al.*, 2025] Ziyi Zhang, Nicolas Roussel, and Wenzel Jakob. Many-worlds inverse rendering. *ACM Trans. Graph.*, 45(1), October 2025.